Docker Interview Questions & Answers

DevOps Engineer (3 to 5 Years Experience)

Q1. How do you optimize a Docker image from 2GB to 200MB?

- Use a minimal base image such as alpine, distroless, or scratch instead of a full OS image.
- Use multi-stage builds to separate build-time dependencies from the final runtime image.
- **Remove build tools, package caches, and temporary files in the same RUN layer they were created.**
  - e.g. *RUN apt-get update && apt-get install -y build-essential && ... && rm -rf /var/lib/apt/lists/**
- Combine multiple RUN commands into one to reduce the number of layers.
- Use .dockerignore to exclude unnecessary files (node_modules, .git, logs) from the build context.
- Avoid installing recommended/suggested packages (--no-install-recommends).
- Use language-specific slim images (python:3.x-slim, node:alpine) where possible.
- Analyze layers with tools like dive or docker history to find bloat.
- Strip debug symbols and avoid copying unnecessary documentation, man pages, or test files.

Q2. How do multi-stage builds work, and when would you use them?

- Multi-stage builds use multiple FROM statements in a single Dockerfile, each starting a new build stage.
- Each stage can use a different base image; artifacts are selectively copied from one stage to another using COPY --from=stage_name.
- Build-time dependencies (compilers, SDKs, dev tools) live only in earlier stages and are never shipped in the final image.
- The final stage typically uses a minimal runtime base image (alpine, distroless) containing only the compiled binary/application.
- **Use cases:**
  - Compiling Go/Java/C++ applications where the build toolchain is large but the binary is small.
  - Running tests or linting in an intermediate stage without polluting the production image.
  - Building frontend assets (npm build) and then copying only the static files into an nginx image.

Q3. How do you reduce Docker build time in CI/CD?

- Order Dockerfile instructions from least to most frequently changing, so Docker's layer cache is reused effectively.
- Copy dependency manifests (package.json, requirements.txt) and install dependencies before copying full source code.
- Use BuildKit (DOCKER_BUILDKIT=1) for parallel stage builds and improved caching.
- Use registry-based cache (--cache-from / --cache-to) in CI pipelines so cache persists across ephemeral runners.
- Use multi-stage builds to avoid rebuilding unrelated stages.
- Use smaller base images to reduce pull and layer-extraction time.

- Leverage CI-native Docker layer caching (e.g. GitHub Actions cache, GitLab Docker cache, buildx cache).
- Build only what changed using monorepo-aware build triggers.
- Use parallel builds for independent services/images.

Q4. How do you handle image versioning and tagging in production?

- Avoid relying on the 'latest' tag in production; it is mutable and not traceable.
- Use semantic versioning (e.g. myapp:1.4.2) tied to application releases.
- Tag images with the Git commit SHA or CI build number for full traceability back to source.
- Use immutable tags: once pushed, a version tag should never be overwritten.
- Maintain environment-specific tags or promote the same image across environments (dev → staging → prod) rather than rebuilding.
- Use a private registry with retention/cleanup policies to manage old images.
- Sign images (e.g. Docker Content Trust / cosign) to verify authenticity before deployment.

Q5. How do you scan Docker images for vulnerabilities?

- Use dedicated scanning tools such as Trivy, Grype, Docker Scout, Snyk, or Clair.
- Integrate scanning into the CI/CD pipeline so builds fail on critical/high vulnerabilities.
- Scan both OS-level packages and application dependencies (language libraries).
- Continuously rescan images already in the registry, since new CVEs are discovered after a build.
- Use minimal/distroless base images to reduce the overall vulnerability surface to scan.
- Combine scanning with policy-as-code (e.g. OPA/Conftest) to enforce organizational security gates.
- Review scan reports for fixable vulnerabilities and patch base images regularly.

Q6. A container keeps restarting. How would you troubleshoot it?

- Check container status and restart count: docker ps -a / docker inspect <container>.
- View logs: docker logs <container> (and docker logs --previous for the last failed run).
- Check the restart policy (always, on-failure, unless-stopped) to confirm expected behavior.
- Inspect exit code via docker inspect --format='{{.State.ExitCode}}' to identify the failure type.
- **Look for common causes:**
  - Application crash due to missing environment variables or misconfiguration.
  - Health check failing (HEALTHCHECK marking the container unhealthy).
  - Out-of-memory kill (exit code 137) due to resource limits being too low.
  - Port conflicts or failed dependency connections (database not reachable).
- Run the image interactively (docker run -it --entrypoint /bin/sh image) to debug manually.
- Check orchestrator events (kubectl describe pod) if running under Kubernetes.

Q7. A container exits immediately after startup. What would you check?

- Check the main process: containers exit when PID 1 exits, so confirm the CMD/ENTRYPOINT process isn't a short-lived script.

- Run docker logs <container> to see the application's last output/error before exit.
- Verify CMD/ENTRYPOINT syntax in the Dockerfile — a typo or wrong path causes immediate failure.
- Check the exit code with docker inspect to distinguish between a clean exit (0) and a crash.
- Confirm required environment variables, config files, or mounted volumes are present.
- Check if the process is meant to run in the foreground; many services daemonize by default and exit when run in a container.
- Try overriding the entrypoint to drop into a shell for manual investigation: docker run -it --entrypoint /bin/sh image.

Q8. How do you debug a container in production?

- Use docker exec -it <container> /bin/sh (or bash) to get a shell inside a running container.
- Use docker logs -f <container> to stream logs in real time.
- Use docker inspect to check configuration, mounts, network settings, and resource limits.
- Use docker stats to monitor live CPU, memory, and network usage.
- For distroless/minimal images without a shell, use ephemeral debug containers (e.g. docker debug, or kubectl debug in Kubernetes) that attach a toolbox image to the target container's namespace.
- Check centralized logging/monitoring (ELK, Loki, Datadog, Prometheus/Grafana) instead of relying only on local logs.
- Reproduce the issue in a staging environment with the same image rather than debugging live production where possible.

Q9. What is the difference between CMD and ENTRYPOINT with real examples?

- ENTRYPOINT defines the fixed, main executable for the container; CMD provides default arguments that can be overridden at runtime.
- If only CMD is set, it can be fully replaced by arguments passed to docker run.
- If only ENTRYPOINT is set, runtime arguments are appended to it rather than replacing it.
- When both are used together, CMD supplies default arguments to ENTRYPOINT, and these defaults can still be overridden.
- **Example using CMD alone:**
  - Dockerfile: CMD ["nginx", "-g", "daemon off;"] → docker run myimage echo hi replaces the whole command with 'echo hi'.
- **Example using ENTRYPOINT + CMD:**
  - Dockerfile: ENTRYPOINT ["python", "app.py"] and CMD ["--env=prod"] → docker run myimage runs 'python app.py --env=prod'; docker run myimage --env=dev runs 'python app.py --env=dev'.
- Use ENTRYPOINT when the image should always run as a specific executable (e.g. a CLI tool); use CMD alone when the default command should be easily replaceable.

Q10. How do you pass runtime arguments to a container?

- Pass environment variables with -e KEY=VALUE or --env-file path/to/.env.
- Pass command-line arguments after the image name: docker run myimage --port=8080 (appended to ENTRYPOINT, or replaces CMD if no ENTRYPOINT is set).

- Use Docker Compose 'environment' or 'env_file' sections for multi-container setups.
- Use build-time variables (ARG) in the Dockerfile for values needed during image build, as opposed to runtime ENV values.
- In Kubernetes, pass arguments via the 'args' and 'env' fields in the Pod spec, or via ConfigMaps/Secrets mounted as env vars or files.

Q11. Volumes vs Bind Mounts — which would you choose and why?

- **Volumes are managed entirely by Docker, stored under Docker's storage area, and are the preferred mechanism for persisting data in production.**
  - Portable across environments, easier to back up, and decoupled from host directory structure.
  - Work well with named-volume drivers for network storage (NFS, cloud block storage).
- **Bind mounts map a specific host filesystem path directly into the container.**
  - Useful in local development for live code reloading, since changes on the host are reflected instantly.
  - Tightly coupled to the host's directory layout, which makes them less portable and a potential security risk if the wrong host path is exposed.
- Recommendation: use volumes for production data persistence (databases, uploads); use bind mounts mainly for local development workflows.

Q12. How do you persist database data in Docker?

- Attach a named volume to the database's data directory (e.g. -v pgdata:/var/lib/postgresql/data).
- Define the volume in docker-compose.yml so it survives 'docker compose down' (without -v) and container recreation.
- Avoid storing data inside the container's writable layer, since that data is lost when the container is removed.
- For production, prefer external/managed storage backends (cloud block storage, NFS) via volume drivers for durability and backup support.
- Set up regular backup jobs (pg_dump, mysqldump, snapshotting the volume) independent of the container lifecycle.
- In orchestrated environments, use PersistentVolumes and PersistentVolumeClaims (Kubernetes) to decouple storage from individual pods.

Q13. How does Docker networking work internally?

- Docker uses Linux kernel features — network namespaces, virtual ethernet (veth) pairs, and Linux bridges — to isolate and connect containers.
- By default, Docker creates a bridge network (docker0) on the host; each container gets its own network namespace and a veth pair connecting it to the bridge.
- iptables rules manage NAT and port forwarding, allowing container ports to be mapped to host ports (-p hostPort:containerPort).
- Docker's embedded DNS server allows containers on the same user-defined network to resolve each other by container/service name.
- For multi-host communication, Docker uses overlay networks built on VXLAN tunneling, allowing containers on different hosts to communicate as if on the same network.

- Network drivers (bridge, host, overlay, macvlan, none) are pluggable, allowing different networking models depending on the deployment.

Q14. Bridge vs Host vs Overlay networks — when would you use each?

- **Bridge network: the default mode; isolates containers in their own network namespace with NAT to the host.**
  - Best for single-host setups where containers need isolation but also need to talk to each other via a private network.
- **Host network: the container shares the host's network namespace directly, with no isolation.**
  - Best for performance-sensitive workloads needing minimal network overhead, but it gives up port isolation and increases security risk.
- **Overlay network: spans multiple Docker hosts, enabling containers on different machines to communicate as if on one network.**
  - Best for multi-host orchestration (Docker Swarm, multi-node clusters) where services must discover and reach each other across nodes.

Q15. How do containers communicate across multiple hosts?

- Overlay networks (e.g. Docker Swarm overlay, Kubernetes CNI plugins like Calico/Flannel/Cilium) create a virtual network spanning multiple hosts using VXLAN or similar encapsulation.
- A distributed key-value store (e.g. etcd, or Swarm's built-in raft store) maintains network state and service discovery information across nodes.
- Service discovery and built-in DNS allow containers to resolve each other by service name regardless of which physical host they run on.
- Ingress/load-balancing layers (Swarm routing mesh, Kubernetes Services/Ingress) route external and inter-service traffic to the correct backend container.
- Underlying physical/network requirements include open ports between hosts for VXLAN traffic and consistent firewall/security group rules across the cluster.

Q16. How do you secure Docker containers in production?

- Run containers as a non-root user (see Q17).
- Use minimal base images (alpine, distroless) to shrink the attack surface.
- Scan images for vulnerabilities before deployment and continuously after (see Q5).
- Set resource limits (CPU, memory) to prevent a compromised or runaway container from impacting the host.
- Use read-only root filesystems where possible (--read-only) and mount only the specific writable paths needed.
- Drop unnecessary Linux capabilities (--cap-drop=ALL, add back only what's required) and avoid --privileged mode.
- Keep the Docker engine and host OS patched and up to date.
- Use network segmentation and firewall rules to restrict container-to-container and container-to-internet traffic.
- Enable and monitor audit logging for container and daemon activity.
- Sign and verify images using Docker Content Trust or cosign before deployment.

Q17. Why shouldn't containers run as root?

- By default, the container's root user maps to UID 0 on the host (unless user namespaces remap it), so a container breakout can grant root access on the host itself.
- Running as root inside the container gives a compromised application unnecessary privileges to modify files, install packages, or escalate further.
- It violates the principle of least privilege — most applications never need root to function.
- Mitigation: create and switch to a dedicated non-root user in the Dockerfile (USER directive), and use Kubernetes securityContext (runAsNonRoot: true, runAsUser) to enforce this at the orchestrator level.

Q18. How do you securely manage secrets and environment variables?

- Never hard-code secrets (API keys, passwords, tokens) in the Dockerfile or bake them into image layers.
- Avoid passing secrets via plain environment variables when possible, since they can leak through docker inspect, process listings, or logs.
- Use dedicated secrets managers: Docker Swarm secrets, Kubernetes Secrets (ideally encrypted at rest), HashiCorp Vault, AWS Secrets Manager, or Azure Key Vault.
- Use build-time secret mounting (--secret flag with BuildKit) instead of ARG/ENV for credentials needed only during build.
- Mount secrets as files (tmpfs-backed) rather than environment variables where the application supports it, reducing exposure.
- Rotate secrets regularly and apply least-privilege access policies to who/what can retrieve them.
- Exclude .env files and credentials from version control and from the Docker build context (.dockerignore).

Q19. How do you minimize the attack surface of a Docker image?

- Start from minimal or distroless base images instead of full-featured OS images.
- Use multi-stage builds so build tools and dependencies never reach the final runtime image.
- Install only the packages strictly required for the application to run.
- Remove package managers, shells, and debugging tools from production images where feasible.
- Run as a non-root user with dropped Linux capabilities.
- Keep the image's writable filesystem read-only at runtime where possible.
- Regularly rebuild images to pick up base-image and dependency security patches.
- Continuously scan for vulnerabilities and remove unused/deprecated dependencies.
- Avoid embedding secrets, SSH keys, or credentials anywhere in the image layers.